UNIVERSIDAD PRIVADA FRANZ TAMAYO

Facultad de Ingeniería · Ingeniería de Sistemas

Testing y Aseguramiento de la Calidad (TQA)

EXAMEN PRÁCTICO – HITO 3

Caja negra · Caja blanca · Gherkin · Bugs · Git y GitHub

Aplicación bajo prueba: SauceDemo (Swag Labs)

Estudiante:	Juan Gabriel Ibañez Copatiti
Modalidad:	Individual
Repositorio:	https://github.com/aJuanga/Examen-Hito-3-TQA
Fecha:	20 de octubre de 2026
Lugar:	La Paz – Bolivia

1. Introducción

El presente informe documenta el Examen Práctico del Hito 3 de la materia Testing y Aseguramiento de la Calidad. El trabajo se realizó sobre SauceDemo (Swag Labs), una tienda en línea de demostración disponible en https://www.saucedemo.com/, que ofrece inicio de sesión, catálogo de productos, carrito de compras y un proceso de checkout en tres pasos.

Sobre esta aplicación se diseñaron y ejecutaron cinco casos de prueba de caja negra, se documentaron los defectos reproducibles encontrados, se analizó un fragmento de código Python con técnicas de caja blanca y se redactaron escenarios en Gherkin. Todo el trabajo quedó versionado en un repositorio individual de GitHub y organizado con Issues y un Project Board.

2. Objetivos

2.1 Objetivo general

Aplicar las técnicas de prueba vistas en la materia sobre una aplicación real de demostración, documentando el diseño, la ejecución y los hallazgos, y organizando el trabajo con Git y GitHub.

2.2 Objetivos específicos

- Diseñar y ejecutar cinco casos de prueba funcionales de caja negra sobre SauceDemo, registrando su resultado.
- Documentar los defectos reproducibles con pasos, resultado esperado y actual, severidad y evidencia.
- Identificar las decisiones y condiciones del código validar_compra y determinar el mínimo de casos para el criterio de cobertura elegido.
- Describir el comportamiento de la aplicación mediante escenarios Gherkin.
- Organizar archivos, tareas y defectos en un repositorio de GitHub usando comandos de Git, Issues y un Project Board.

3. Marco teórico

Pruebas de caja negra. Evalúan el sistema desde la perspectiva del usuario, comparando el comportamiento observado con el esperado sin conocer el código interno. Un caso de prueba define objetivo, precondiciones, datos, pasos y resultado esperado, y se ejecuta para obtener un resultado PASS o FAIL.

Pruebas de caja blanca. Analizan la estructura interna del código. La cobertura de sentencias exige ejecutar cada sentencia al menos una vez; la cobertura de decisiones exige que cada decisión tome los valores verdadero y falso; y la cobertura de condiciones exige que cada condición simple sea verdadera y falsa al menos una vez.

Gherkin y BDD. Gherkin es un lenguaje estructurado que describe el comportamiento esperado mediante las palabras clave Feature, Scenario, Given, When y Then. Facilita que negocio, desarrollo y QA compartan los mismos criterios de aceptación.

Gestión de defectos. Un reporte de bug debe permitir que otra persona reproduzca el problema: incluye título, descripción, precondiciones, pasos, resultado esperado y actual, severidad, prioridad, ambiente y evidencia.

Git y GitHub. Git es un sistema de control de versiones distribuido que registra cada cambio mediante commits. GitHub aloja el repositorio remoto y agrega herramientas de colaboración como Issues y Projects para planificar y dar seguimiento al trabajo.

4. Recursos utilizados

Recurso	Descripción / uso
Aplicación bajo prueba	SauceDemo – Swag Labs (https://www.saucedemo.com/)
Usuarios de demostración	standard_user, locked_out_user y problem_user (contraseña secret_sauce)
Navegador	Chromium 141 · resolución 1280x720
Plantillas de la materia	Plantilla de casos de prueba (diseño y ejecución) y plantilla de bug report
Python 3, pytest y coverage.py	Ejecución de los casos de caja blanca y medición de cobertura de ramas
Git	Control de versiones desde terminal: init, add, commit, remote, push
GitHub	Repositorio público, Issues con etiquetas y Project Board
Markdown	Documentación del repositorio (README, casos, bugs, análisis y escenarios)

5. Pruebas de caja negra

Se seleccionaron cinco funcionalidades representativas de SauceDemo: inicio de sesión exitoso, bloqueo de un usuario deshabilitado, gestión del carrito, ordenamiento del catálogo y llenado de datos en el checkout. Los casos se ejecutaron el 20/10/2026 y cada uno se registró en la plantilla de la materia.

5.1 Resumen de ejecución

ID	Título	Módulo	Tipo	Prioridad	Resultado	Bug
TC-001	Inicio de sesión exitoso con usuario estándar	Inicio de sesión	Funcional	Alta	**PASS**	N/A
TC-002	Bloqueo de acceso a un usuario deshabilitado	Inicio de sesión	Negativa	Alta	**PASS**	N/A
TC-003	Agregar y eliminar productos del carrito	Carrito de compras	Funcional	Media	**PASS**	N/A
TC-004	Ordenar productos por precio de menor a mayor	Catálogo – ordenamiento	Funcional	Media	**FAIL**	BUG-002
TC-005	Completar los datos del checkout con información válida	Checkout	Funcional	Alta	**FAIL**	BUG-001

De los cinco casos, tres pasaron y dos fallaron. Los dos fallos aparecen con el usuario problem_user y no con standard_user, lo que se verificó repitiendo el mismo flujo con ambos usuarios. Cada fallo dio origen a un reporte de bug.

5.2 Diseño y ejecución de los casos

En las páginas siguientes se presenta cada caso en la plantilla de la materia: la Parte 1 contiene el diseño y la Parte 2 el registro de ejecución, con las capturas de evidencia y un comentario de recomendación.

CASOS DE PRUEBA

Plantilla para diseño y ejecución

TQA • Documento base para prácticas e integrador

01 DISEÑAR EL TC Definir qué se quiere comprobar y cómo.	02 EJECUTAR LA PRUEBA Registrar qué ocurrió realmente.

Solo incluye diseño y ejecución. La trazabilidad se incorporará más adelante.

PARTE 1 · PLANTILLA PARA CREAR EL CASO DE PRUEBA

Completar antes de ejecutar la prueba.

ID DEL TC	TC-001	**PRIORIDAD**	Alta
TÍTULO	Inicio de sesión exitoso con usuario estándar	**TIPO**	Funcional
MÓDULO	Inicio de sesión	**AMBIENTE**	QA (sitio de demostración)

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que un usuario registrado y habilitado accede al catálogo de productos al ingresar credenciales válidas.

2. Precondiciones

Estado inicial	Navegador abierto en la página de login de SauceDemo, sin sesión iniciada.
Usuario / permisos	standard_user (usuario de demostración habilitado).
Configuración	Conexión a internet activa. Sin configuraciones adicionales.

3. Datos de prueba

Dato	Usuario / contraseña
Valor	standard_user / secret_sauce
Origen	Ambiente (credenciales publicadas en la página de login)
Observación	Usuario sin restricciones; sirve como línea base para los demás casos.

4. Pasos y resultados esperados

Los pasos deben ser reproducibles: otro tester debería poder ejecutarlos sin pedir aclaraciones.

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir la URL de la aplicación	https://www.saucedemo.com/	Se muestra el formulario de login con los campos Username y Password.
2	Ingresar el usuario en el campo Username	standard_user	El campo muestra el texto ingresado.
3	Ingresar la contraseña en el campo Password	secret_sauce	El campo muestra caracteres ocultos.
4	Presionar el botón Login	—	El sistema redirige a /inventory.html con el título Products.

5	Verificar el catálogo	—	Se listan los 6 productos con nombre, precio y botón Add to cart.

5. Resultado esperado final

CRITERIO DE ÉXITO	El usuario accede a /inventory.html y visualiza el catálogo completo sin mensajes de error.

PARTE 2 · PLANTILLA PARA EJECUTAR LA PRUEBA

Completar durante o inmediatamente después de ejecutar el TC.

ID DEL TC	TC-001	**EJECUTADO POR**	Juan Gabriel Ibañez Copatiti
FECHA	20/10/2026	**HORA**	21:18
AMBIENTE	QA (sitio de demostración)	**VERSIÓN**	SauceDemo – Swag Labs (web)

1. Resultado de la ejecución

☒ PASS	☐ FAIL	☐ BLOCKED	☐ NOT RUN

2. Resultado observado

¿QUÉ OCURRIÓ REALMENTE?	Tras presionar Login, la aplicación redirigió a https://www.saucedemo.com/inventory.html, mostró el título Products y listó los 6 productos con sus precios.

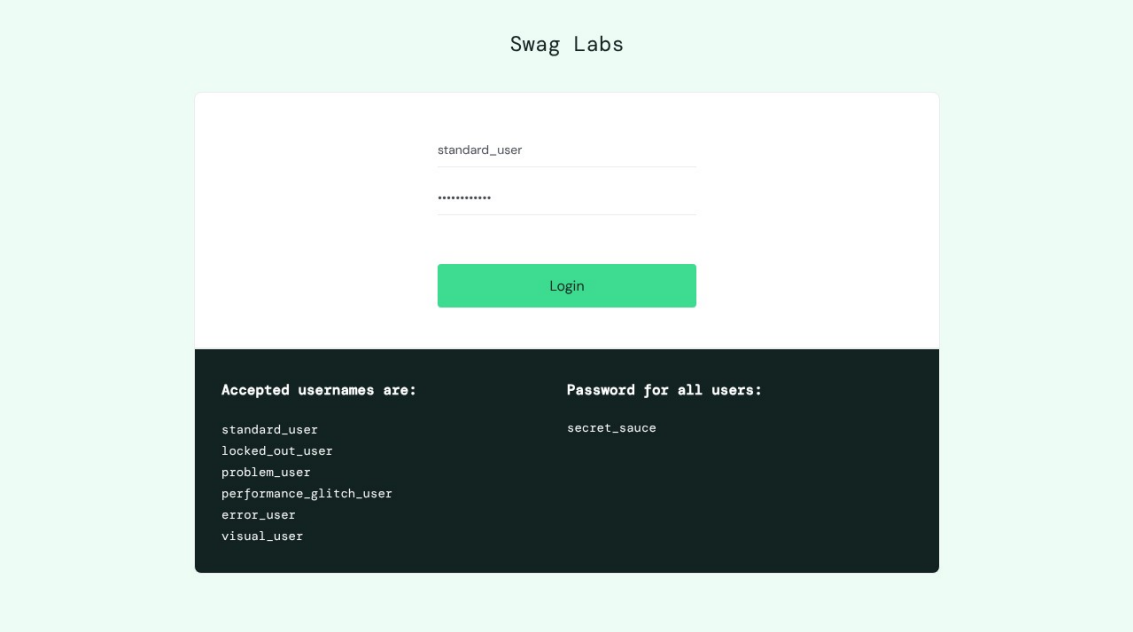
COMPARACIÓN	Coincide con el resultado esperado.

3. Registro de ejecución por paso

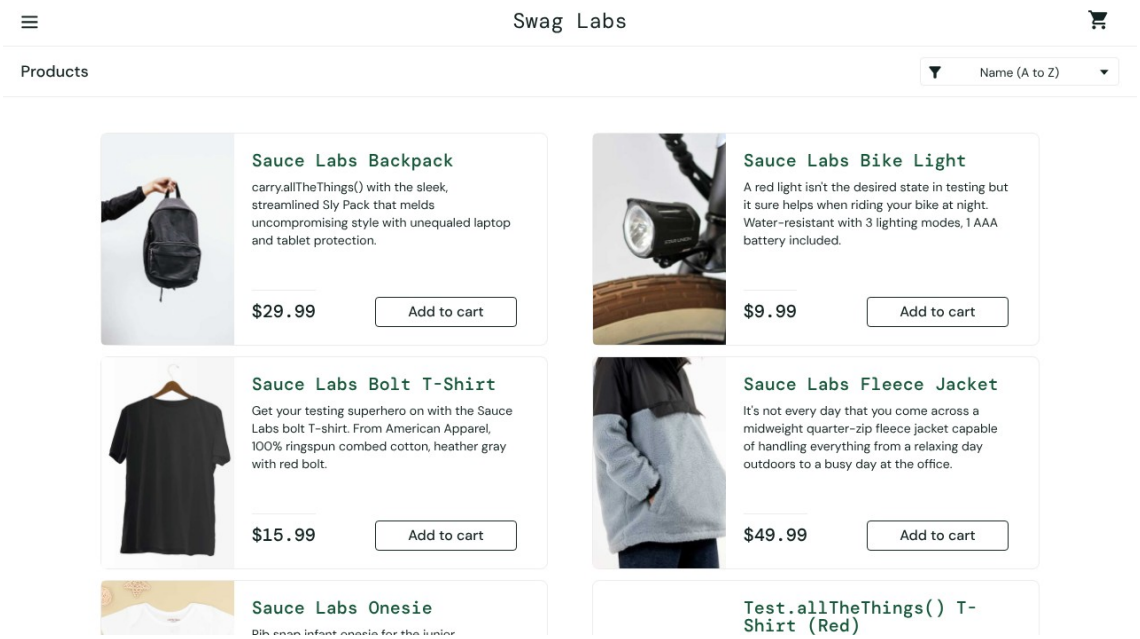
#	Paso ejecutado	Resultado observado	Estado	Observación
1	Abrió la URL	Formulario de login visible	PASS	—
2	Ingresó standard_user	Texto visible en Username	PASS	—
3	Ingresó secret_sauce	Caracteres ocultos	PASS	—
4	Presionó Login	Redirección a /inventory.html	PASS	Respuesta inmediata
5	Revisó el catálogo	6 productos listados	PASS	—

4. Evidencia

Tipo	Captura de pantalla
Nombre o referencia	TC-001_01_credenciales.png TC-001_02_catalogo.png
Descripción	Credenciales cargadas antes del login y catálogo mostrado después del acceso.



TC-001_01_credenciales.png



TC-001_02_catalogo.png

5. Si el resultado es FAIL

DESVIACIÓN ENCONTRADA	N/A

REFERENCIA DEL DEFECTO	N/A

6. Observaciones de ejecución

OBSERVACIONES	Recomendación: agregar a futuro un caso de login con contraseña incorrecta para el mismo usuario y un caso de cierre de sesión, de modo que el módulo quede cubierto en sus flujos principales.

Reglas de uso

- El TC se diseña primero; el resultado observado se registra durante la ejecución.
- El resultado esperado debe estar definido antes de ejecutar.
- PASS = coincide; FAIL = diferencia verificable; BLOCKED = no se pudo ejecutar; NOT RUN = aún no ejecutado.
- Las evidencias deben permitir entender el resultado.
- El TC debe ser claro, concreto y reproducible por otro tester.

CASOS DE PRUEBA

Plantilla para diseño y ejecución

TQA • Documento base para prácticas e integrador

01 DISEÑAR EL TC Definir qué se quiere comprobar y cómo.	02 EJECUTAR LA PRUEBA Registrar qué ocurrió realmente.

Solo incluye diseño y ejecución. La trazabilidad se incorporará más adelante.

PARTE 1 · PLANTILLA PARA CREAR EL CASO DE PRUEBA

Completar antes de ejecutar la prueba.

ID DEL TC	TC-002	**PRIORIDAD**	Alta
TÍTULO	Bloqueo de acceso a un usuario deshabilitado	**TIPO**	Negativa
MÓDULO	Inicio de sesión	**AMBIENTE**	QA (sitio de demostración)

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que un usuario bloqueado no puede ingresar al sistema y que se le informa el motivo.

2. Precondiciones

Estado inicial	Navegador abierto en la página de login, sin sesión iniciada.
Usuario / permisos	locked_out_user (usuario de demostración bloqueado).
Configuración	Ninguna.

3. Datos de prueba

Dato	Usuario / contraseña
Valor	locked_out_user / secret_sauce
Origen	Ambiente (credenciales publicadas en la página de login)
Observación	La contraseña es correcta; lo que se valida es el estado bloqueado de la cuenta.

4. Pasos y resultados esperados

Los pasos deben ser reproducibles: otro tester debería poder ejecutarlos sin pedir aclaraciones.

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir la URL de la aplicación	https://www.saucedemo.com/	Se muestra el formulario de login.
2	Ingresar el usuario bloqueado	locked_out_user	El campo muestra el texto ingresado.
3	Ingresar la contraseña	secret_sauce	El campo muestra caracteres ocultos.
4	Presionar el botón Login	—	El sistema no permite el acceso y permanece en la página de login.
5	Verificar el mensaje	—	Se muestra: "Epic sadface: Sorry, this user has been

			locked out."

5. Resultado esperado final

CRITERIO DE ÉXITO	El acceso se rechaza y aparece un mensaje claro que indica que el usuario está bloqueado.

PARTE 2 · PLANTILLA PARA EJECUTAR LA PRUEBA

Completar durante o inmediatamente después de ejecutar el TC.

ID DEL TC	TC-002	**EJECUTADO POR**	Juan Gabriel Ibañez Copatiti
FECHA	20/10/2026	**HORA**	21:20
AMBIENTE	QA (sitio de demostración)	**VERSIÓN**	SauceDemo – Swag Labs (web)

1. Resultado de la ejecución

☒ PASS	☐ FAIL	☐ BLOCKED	☐ NOT RUN

2. Resultado observado

¿QUÉ OCURRIÓ REALMENTE?	La aplicación se mantuvo en la página de login, marcó los campos en rojo y mostró el mensaje "Epic sadface: Sorry, this user has been locked out."

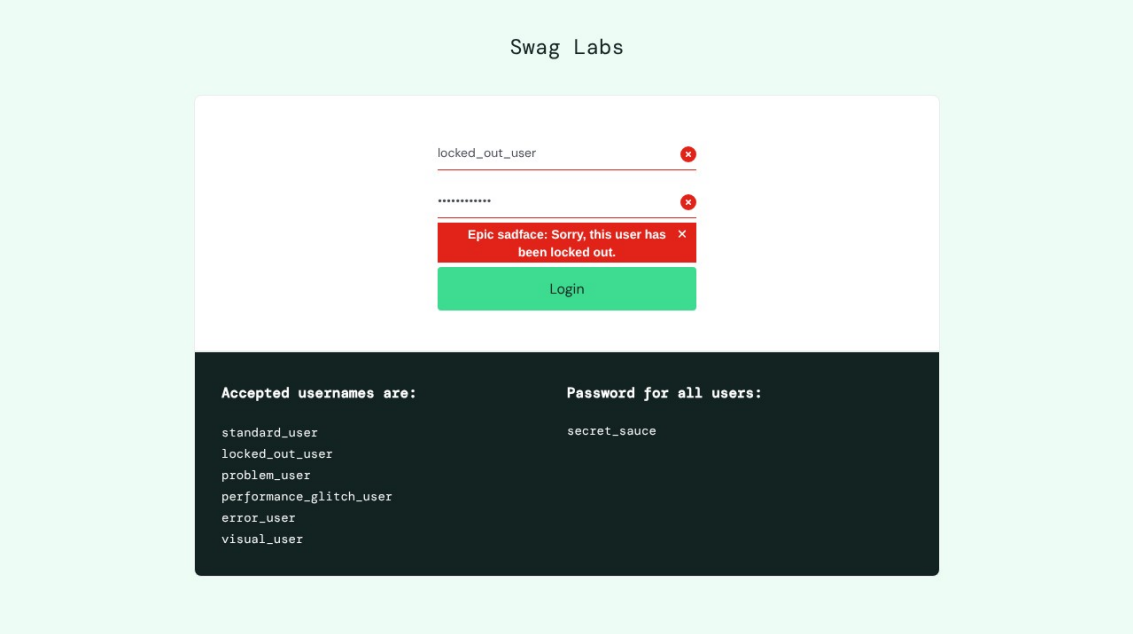
COMPARACIÓN	Coincide con el resultado esperado.

3. Registro de ejecución por paso

#	Paso ejecutado	Resultado observado	Estado	Observación
1	Abrió la URL	Formulario de login visible	PASS	—
2	Ingresó locked_out_user	Texto visible	PASS	—
3	Ingresó secret_sauce	Caracteres ocultos	PASS	—
4	Presionó Login	Permaneció en el login	PASS	—
5	Revisó el mensaje	Mensaje de usuario bloqueado	PASS	Texto en inglés

4. Evidencia

Tipo	Captura de pantalla
Nombre o referencia	TC-002_01_usuario_bloqueado.png
Descripción	Mensaje de error mostrado al intentar ingresar con el usuario bloqueado.



TC-002_01_usuario_bloqueado.png

5. Si el resultado es FAIL

DESVIACIÓN ENCONTRADA	N/A

REFERENCIA DEL DEFECTO	N/A

6. Observaciones de ejecución

OBSERVACIONES	Recomendación: el mensaje cumple su función, pero conviene que indique al usuario qué hacer a continuación (por ejemplo, contactar al administrador) y que se ofrezca en el idioma del usuario.

Reglas de uso

- El TC se diseña primero; el resultado observado se registra durante la ejecución.
- El resultado esperado debe estar definido antes de ejecutar.
- PASS = coincide; FAIL = diferencia verificable; BLOCKED = no se pudo ejecutar; NOT RUN = aún no ejecutado.
- Las evidencias deben permitir entender el resultado.
- El TC debe ser claro, concreto y reproducible por otro tester.

CASOS DE PRUEBA

Plantilla para diseño y ejecución

TQA • Documento base para prácticas e integrador

01 DISEÑAR EL TC Definir qué se quiere comprobar y cómo.	02 EJECUTAR LA PRUEBA Registrar qué ocurrió realmente.

Solo incluye diseño y ejecución. La trazabilidad se incorporará más adelante.

PARTE 1 · PLANTILLA PARA CREAR EL CASO DE PRUEBA

Completar antes de ejecutar la prueba.

ID DEL TC	TC-003	**PRIORIDAD**	Media
TÍTULO	Agregar y eliminar productos del carrito	**TIPO**	Funcional
MÓDULO	Carrito de compras	**AMBIENTE**	QA (sitio de demostración)

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que el carrito refleja correctamente los productos agregados y eliminados, y que el contador del ícono se actualiza.

2. Precondiciones

Estado inicial	Sesión iniciada con standard_user en el catálogo, con el carrito vacío.
Usuario / permisos	standard_user.
Configuración	Ninguna.

3. Datos de prueba

Dato	Productos
Valor	Sauce Labs Backpack ($29.99) y Sauce Labs Bike Light ($9.99)
Origen	Preparado (catálogo de la aplicación)
Observación	Se eligen dos productos para verificar el contador antes y después de eliminar uno.

4. Pasos y resultados esperados

Los pasos deben ser reproducibles: otro tester debería poder ejecutarlos sin pedir aclaraciones.

#	Paso / acción	Dato utilizado	Resultado esperado
1	Presionar Add to cart en Sauce Labs Backpack	Backpack	El botón cambia a Remove y el contador muestra 1.
2	Presionar Add to cart en Sauce Labs Bike Light	Bike Light	El contador del carrito muestra 2.
3	Abrir el carrito desde el ícono superior	—	Se listan ambos productos con su precio.
4	Presionar Remove en Sauce Labs Backpack	Backpack	El producto desaparece del carrito.

5	Verificar el contador del carrito	—	El contador muestra 1 y solo queda Sauce Labs Bike Light.

5. Resultado esperado final

CRITERIO DE ÉXITO	El carrito y su contador reflejan en todo momento los productos agregados y eliminados.

PARTE 2 · PLANTILLA PARA EJECUTAR LA PRUEBA

Completar durante o inmediatamente después de ejecutar el TC.

ID DEL TC	TC-003	**EJECUTADO POR**	Juan Gabriel Ibañez Copatiti
FECHA	20/10/2026	**HORA**	21:23
AMBIENTE	QA (sitio de demostración)	**VERSIÓN**	SauceDemo – Swag Labs (web)

1. Resultado de la ejecución

☒ PASS	☐ FAIL	☐ BLOCKED	☐ NOT RUN

2. Resultado observado

¿QUÉ OCURRIÓ REALMENTE?	El contador mostró 2 al agregar ambos productos. En el carrito se listaron Sauce Labs Backpack y Sauce Labs Bike Light. Al eliminar el Backpack, solo quedó Bike Light y el contador bajó a 1.

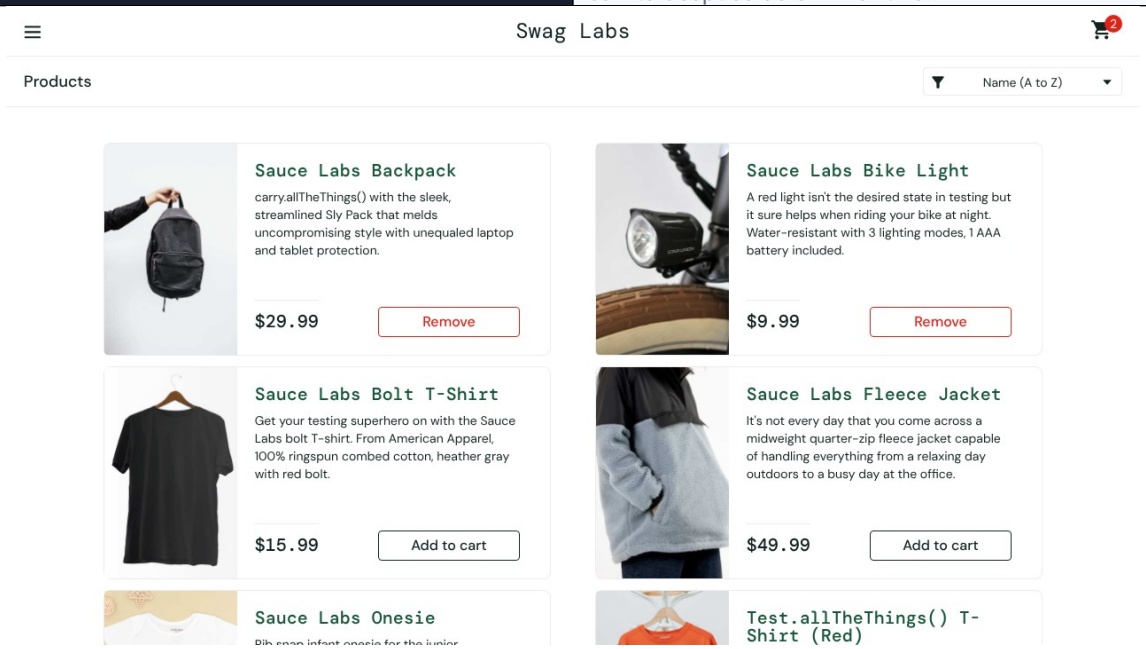
COMPARACIÓN	Coincide con el resultado esperado.

3. Registro de ejecución por paso

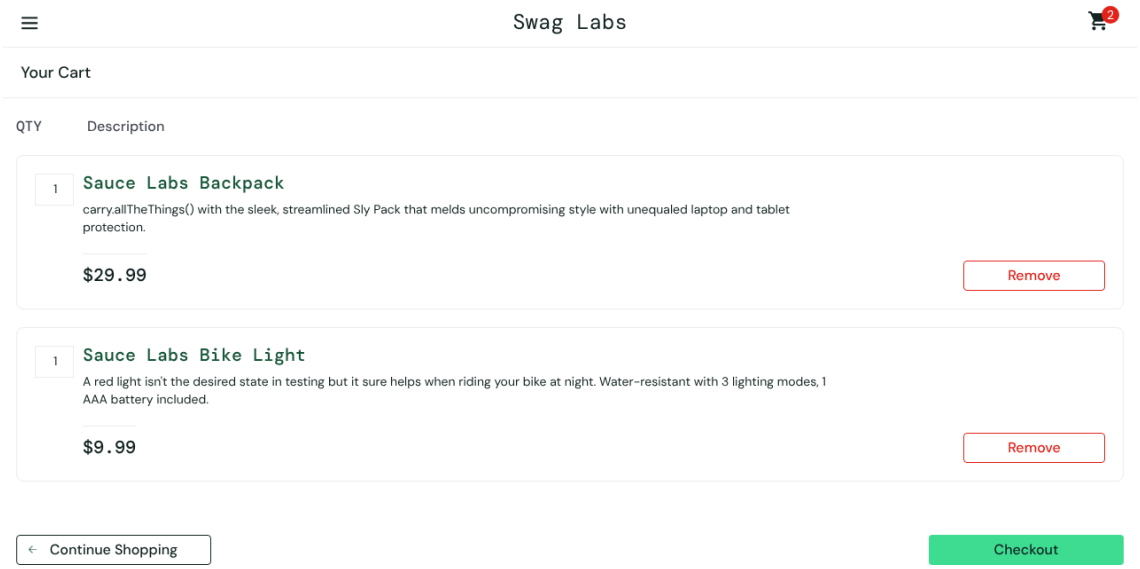
#	Paso ejecutado	Resultado observado	Estado	Observación
1	Agregó Backpack	Botón cambió a Remove	PASS	—
2	Agregó Bike Light	Contador = 2	PASS	—
3	Abrió el carrito	2 productos listados	PASS	—
4	Eliminó Backpack	Producto retirado	PASS	—
5	Revisó el contador	Contador = 1	PASS	—

4. Evidencia

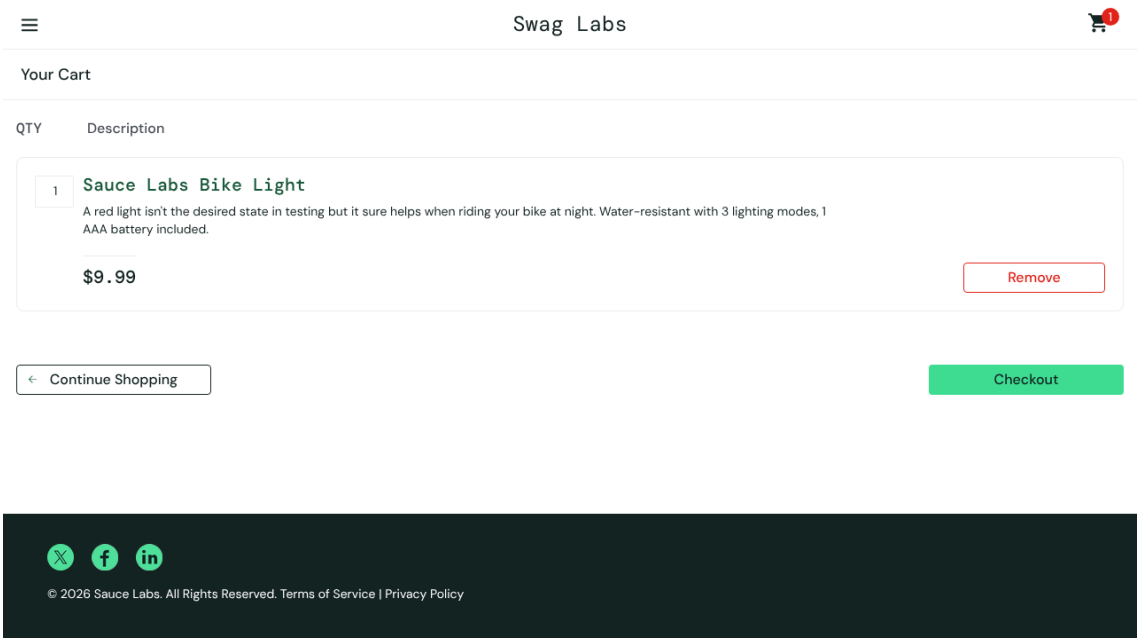
Tipo	Captura de pantalla
Nombre o referencia	TC-003_01_dos_productos.png TC-003_02_carrito.png TC-003_03_producto_eliminado.png
Descripción	Contador con dos productos, carrito con ambos productos y carrito después de eliminar uno.



TC-003_01_dos_productos.png



TC-003_02_carrito.png



TC-003_03_producto_eliminado.png

5. Si el resultado es FAIL

DESVIACIÓN ENCONTRADA	N/A

REFERENCIA DEL DEFECTO	N/A

6. Observaciones de ejecución

OBSERVACIONES	Recomendación: complementar con un caso que verifique que el carrito se conserva al cerrar y volver a iniciar

	sesión, porque es un punto donde suelen aparecer defectos de persistencia.

Reglas de uso

- El TC se diseña primero; el resultado observado se registra durante la ejecución.
- El resultado esperado debe estar definido antes de ejecutar.
- PASS = coincide; FAIL = diferencia verificable; BLOCKED = no se pudo ejecutar; NOT RUN = aún no ejecutado.
- Las evidencias deben permitir entender el resultado.
- El TC debe ser claro, concreto y reproducible por otro tester.

CASOS DE PRUEBA

Plantilla para diseño y ejecución

TQA • Documento base para prácticas e integrador

01 DISEÑAR EL TC Definir qué se quiere comprobar y cómo.	02 EJECUTAR LA PRUEBA Registrar qué ocurrió realmente.

Solo incluye diseño y ejecución. La trazabilidad se incorporará más adelante.

PARTE 1 · PLANTILLA PARA CREAR EL CASO DE PRUEBA

Completar antes de ejecutar la prueba.

ID DEL TC	TC-004	**PRIORIDAD**	Media
TÍTULO	Ordenar productos por precio de menor a mayor	**TIPO**	Funcional
MÓDULO	Catálogo – ordenamiento	**AMBIENTE**	QA (sitio de demostración)

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que el filtro Price (low to high) reordena el catálogo de forma ascendente según el precio.

2. Precondiciones

Estado inicial	Sesión iniciada con problem_user en el catálogo, con el orden por defecto Name (A to Z).
Usuario / permisos	problem_user (usuario de demostración).
Configuración	Ninguna.

3. Datos de prueba

Dato	Opción de ordenamiento
Valor	Price (low to high)
Origen	Preparado (opciones del filtro)
Observación	Orden esperado: $7.99, $9.99, $15.99, $15.99, $29.99, $49.99.

4. Pasos y resultados esperados

Los pasos deben ser reproducibles: otro tester debería poder ejecutarlos sin pedir aclaraciones.

#	Paso / acción	Dato utilizado	Resultado esperado
1	Iniciar sesión con problem_user	problem_user / secret_sauce	Se muestra el catálogo ordenado por nombre.
2	Anotar el orden inicial de precios	—	Orden inicial: $29.99, $9.99, $15.99, $49.99, $7.99, $15.99.
3	Abrir el filtro de ordenamiento	—	Se despliegan las 4 opciones de orden.
4	Seleccionar Price (low to high)	Price (low to high)	El filtro muestra la opción elegida.
5	Verificar el orden de los productos	—	Los precios quedan en orden ascendente, de $7.99 a

			$49.99.

5. Resultado esperado final

CRITERIO DE ÉXITO	Los productos se muestran ordenados de menor a mayor precio y el filtro conserva la opción seleccionada.

PARTE 2 · PLANTILLA PARA EJECUTAR LA PRUEBA

Completar durante o inmediatamente después de ejecutar el TC.

ID DEL TC	TC-004	**EJECUTADO POR**	Juan Gabriel Ibañez Copatiti
FECHA	20/10/2026	**HORA**	21:26
AMBIENTE	QA (sitio de demostración)	**VERSIÓN**	SauceDemo – Swag Labs (web)

1. Resultado de la ejecución

☐ PASS	☒ FAIL	☐ BLOCKED	☐ NOT RUN

2. Resultado observado

¿QUÉ OCURRIÓ REALMENTE?	Al seleccionar Price (low to high), el catálogo no cambió: los precios siguieron en el orden $29.99, $9.99, $15.99, $49.99, $7.99, $15.99 y el filtro volvió a mostrar Name (A to Z).

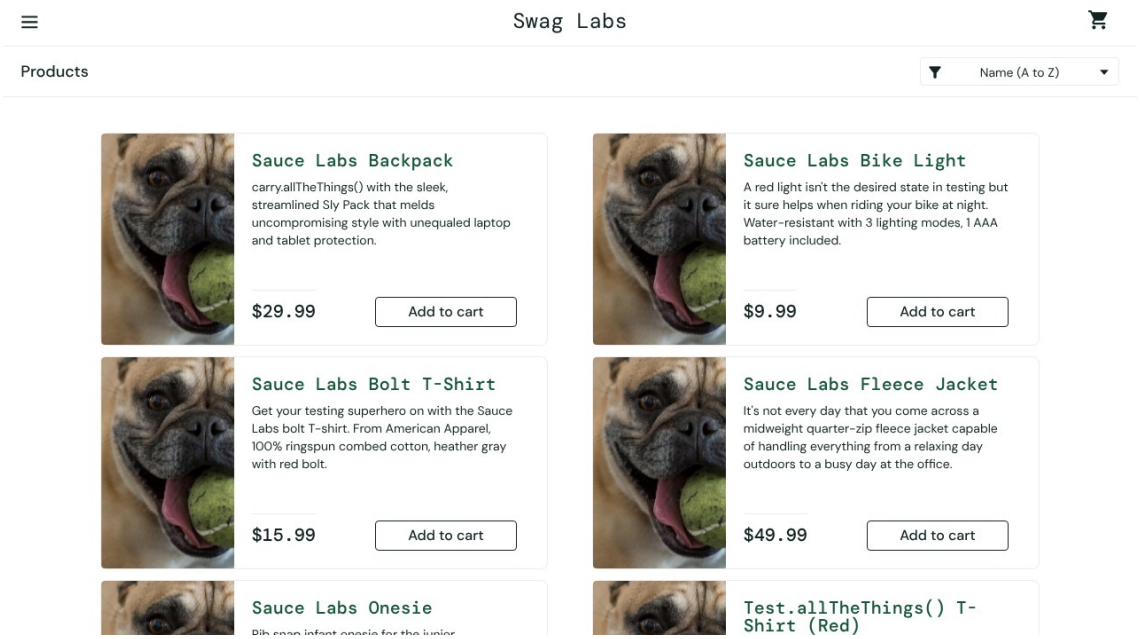
COMPARACIÓN	No coincide. El orden no se aplica y la selección del filtro se revierte. Con standard_user el mismo filtro sí ordena correctamente, por lo que el defecto se presenta con este usuario.

3. Registro de ejecución por paso

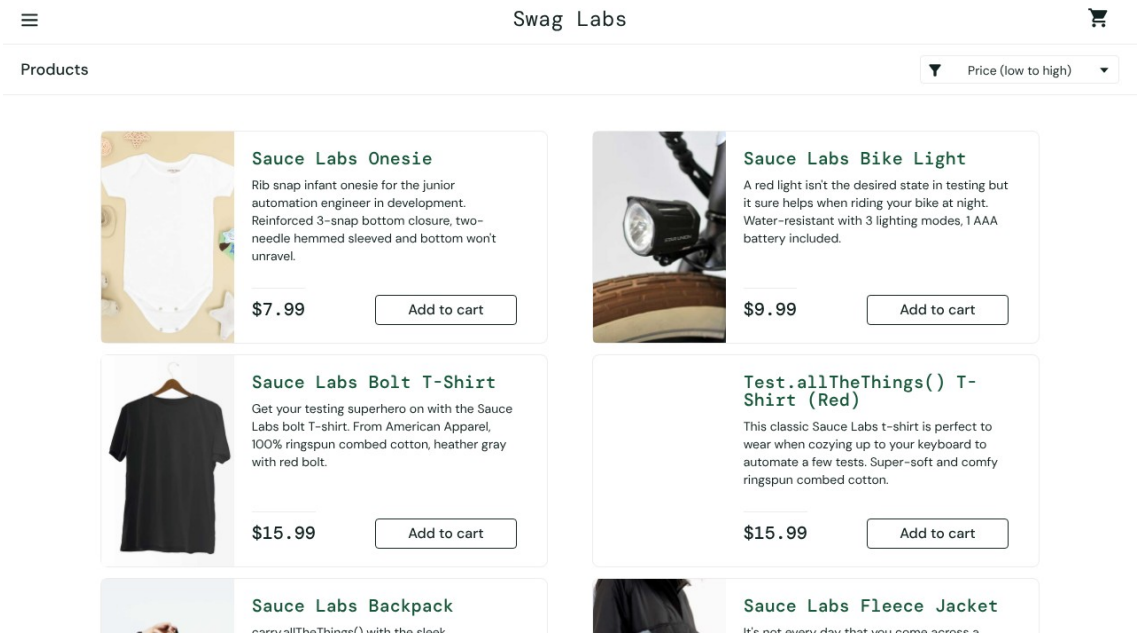
#	Paso ejecutado	Resultado observado	Estado	Observación
1	Inició sesión con problem_user	Catálogo visible	PASS	Todas las imágenes iguales
2	Anotó el orden inicial	Orden por nombre	PASS	—
3	Abrió el filtro	Opciones visibles	PASS	—
4	Seleccionó Price (low to high)	Filtro volvió a Name (A to Z)	**FAIL**	Selección revertida
5	Verificó el orden	Orden sin cambios	**FAIL**	Ver BUG-002

4. Evidencia

Tipo	Captura de pantalla
Nombre o referencia	TC-004_01_orden_no_aplicado.png CTRL_orden_standard_user_ok.png
Descripción	Catálogo de problem_user sin reordenar y, como control, el mismo filtro funcionando con standard_user.



TC-004_01_orden_no_aplicado.png



CTRL_orden_standard_user_ok.png

5. Si el resultado es FAIL

DESVIACIÓN ENCONTRADA	El ordenamiento por precio no se aplica y el filtro vuelve a la opción por defecto.

REFERENCIA DEL DEFECTO	BUG-002

6. Observaciones de ejecución

OBSERVACIONES	Recomendación: corregir el defecto y repetir el caso con las cuatro opciones del filtro (A-Z, Z-A, precio ascendente y descendente) como prueba de regresión. También se

	observó que todos los productos muestran la misma imagen, lo que conviene revisar por separado.

Reglas de uso

- El TC se diseña primero; el resultado observado se registra durante la ejecución.
- El resultado esperado debe estar definido antes de ejecutar.
- PASS = coincide; FAIL = diferencia verificable; BLOCKED = no se pudo ejecutar; NOT RUN = aún no ejecutado.
- Las evidencias deben permitir entender el resultado.
- El TC debe ser claro, concreto y reproducible por otro tester.

CASOS DE PRUEBA

Plantilla para diseño y ejecución

TQA • Documento base para prácticas e integrador

01 DISEÑAR EL TC Definir qué se quiere comprobar y cómo.	02 EJECUTAR LA PRUEBA Registrar qué ocurrió realmente.

Solo incluye diseño y ejecución. La trazabilidad se incorporará más adelante.

PARTE 1 · PLANTILLA PARA CREAR EL CASO DE PRUEBA

Completar antes de ejecutar la prueba.

ID DEL TC	TC-005	**PRIORIDAD**	Alta
TÍTULO	Completar los datos del checkout con información válida	**TIPO**	Funcional
MÓDULO	Checkout	**AMBIENTE**	QA (sitio de demostración)

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que el formulario Checkout: Your Information valida los campos obligatorios y permite continuar al ingresar nombre, apellido y código postal válidos.

2. Precondiciones

Estado inicial	Sesión iniciada con problem_user y un producto en el carrito (Sauce Labs Backpack).
Usuario / permisos	problem_user (usuario de demostración).
Configuración	Ninguna.

3. Datos de prueba

Dato	Nombre / Apellido / Código postal
Valor	Juan / Ibanez / 0201
Origen	Preparado por el tester
Observación	Datos válidos en los tres campos; se omite la tilde del apellido para evitar problemas de codificación.

4. Pasos y resultados esperados

Los pasos deben ser reproducibles: otro tester debería poder ejecutarlos sin pedir aclaraciones.

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir el carrito y presionar Checkout	Sauce Labs Backpack	Se muestra el formulario Checkout: Your Information.
2	Presionar Continue con los campos vacíos	—	Aparece el mensaje Error: First Name is required.
3	Ingresar el nombre en First Name	Juan	El campo First Name muestra Juan.
4	Ingresar el apellido en Last Name	Ibanez	El campo Last Name muestra Ibanez.

5	Ingresar el código postal	0201	El campo Zip/Postal Code muestra 0201.
6	Presionar Continue	—	El sistema avanza a Checkout: Overview con el resumen de la compra.

5. Resultado esperado final

CRITERIO DE ÉXITO	Con los tres campos completos, el sistema avanza al resumen de la compra sin mostrar errores.

PARTE 2 · PLANTILLA PARA EJECUTAR LA PRUEBA

Completar durante o inmediatamente después de ejecutar el TC.

ID DEL TC	TC-005	**EJECUTADO POR**	Juan Gabriel Ibañez Copatiti
FECHA	20/10/2026	**HORA**	21:30
AMBIENTE	QA (sitio de demostración)	**VERSIÓN**	SauceDemo – Swag Labs (web)

1. Resultado de la ejecución

☐ PASS	☒ FAIL	☐ BLOCKED	☐ NOT RUN

2. Resultado observado

¿QUÉ OCURRIÓ REALMENTE?	La validación de campos vacíos funcionó. Sin embargo, al escribir el apellido en Last Name, el texto apareció en First Name (reemplazando Juan por Ibanez) y Last Name quedó vacío. Al presionar Continue se mostró Error: Last Name is required y no fue posible avanzar.

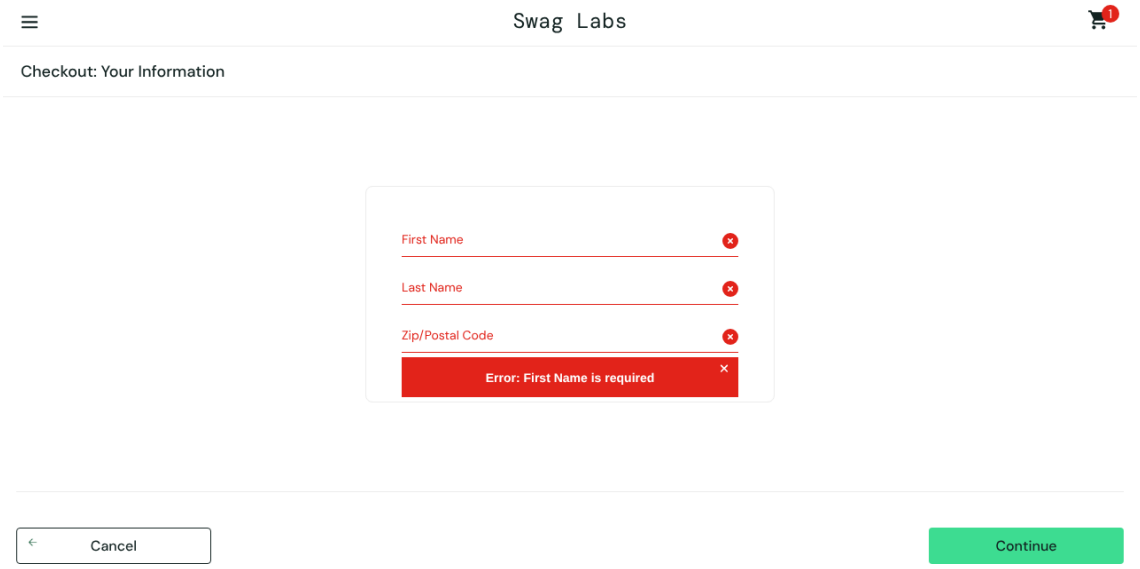
COMPARACIÓN	No coincide. El campo Last Name no recibe el texto ingresado y la compra queda bloqueada en el primer paso del checkout.

3. Registro de ejecución por paso

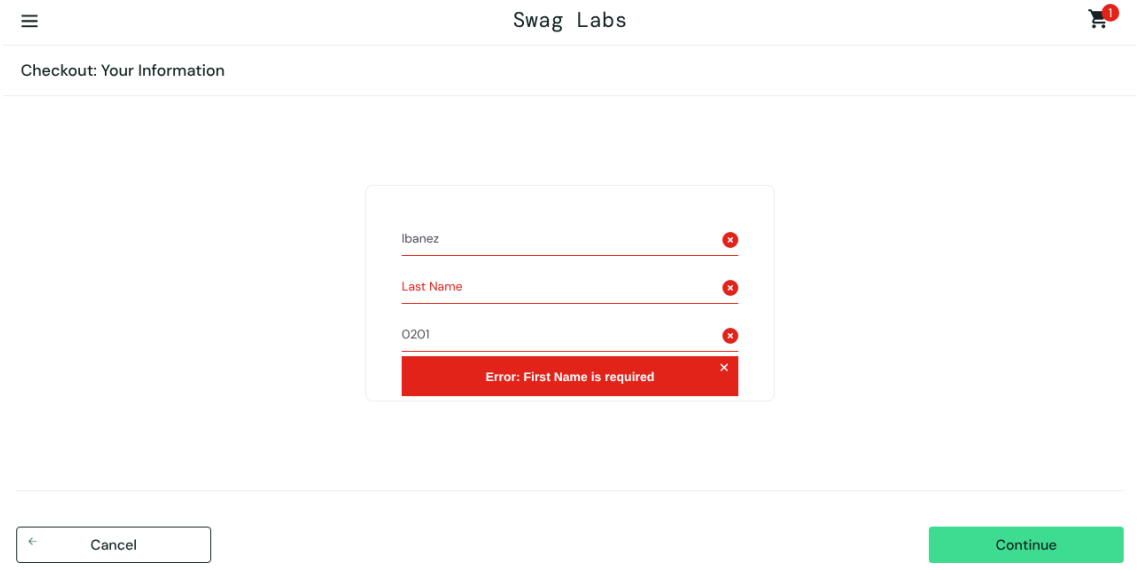
#	Paso ejecutado	Resultado observado	Estado	Observación
1	Abrió el checkout	Formulario visible	PASS	—
2	Presionó Continue vacío	Error: First Name is required	PASS	Validación correcta
3	Ingresó Juan	First Name = Juan	PASS	—
4	Ingresó Ibanez en Last Name	El texto pasó a First Name	**FAIL**	Last Name vacío
5	Ingresó 0201	Código postal correcto	PASS	—
6	Presionó Continue	Error: Last Name is required	**FAIL**	Ver BUG-001

4. Evidencia

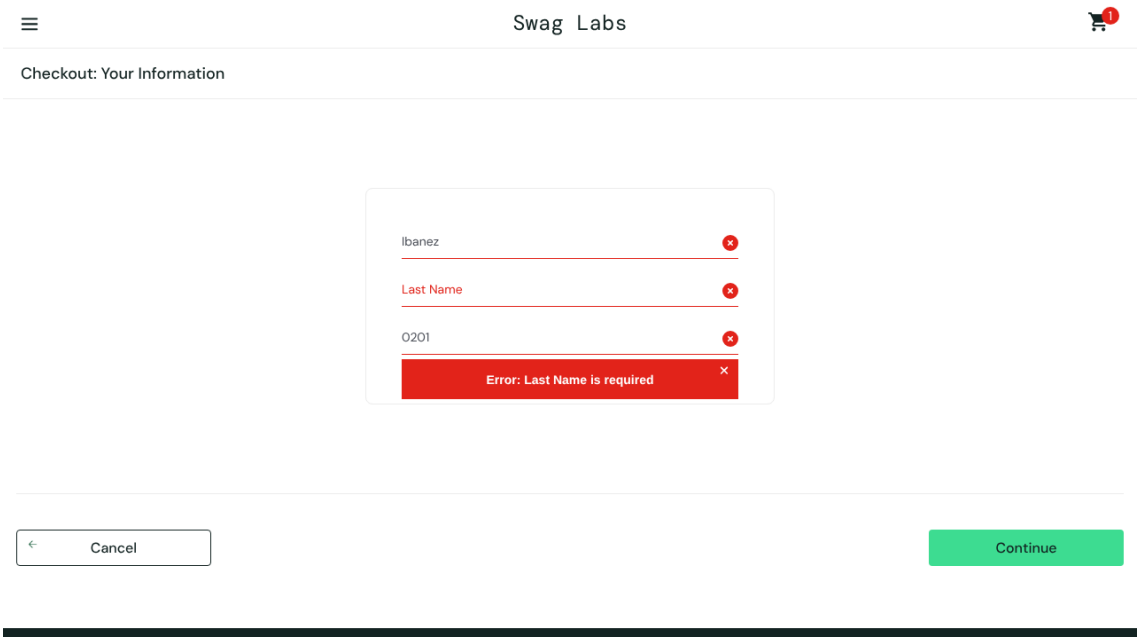
Tipo	Captura de pantalla
Nombre o referencia	TC-005_01_campos_obligatorios.png TC-005_02_apellido_en_nombre.png TC-005_03_error_apellido.png CTRL_checkout_standard_user_ok.png
Descripción	Validación de campos vacíos, apellido escrito en el campo equivocado, error final y, como control, la compra completada con standard_user.



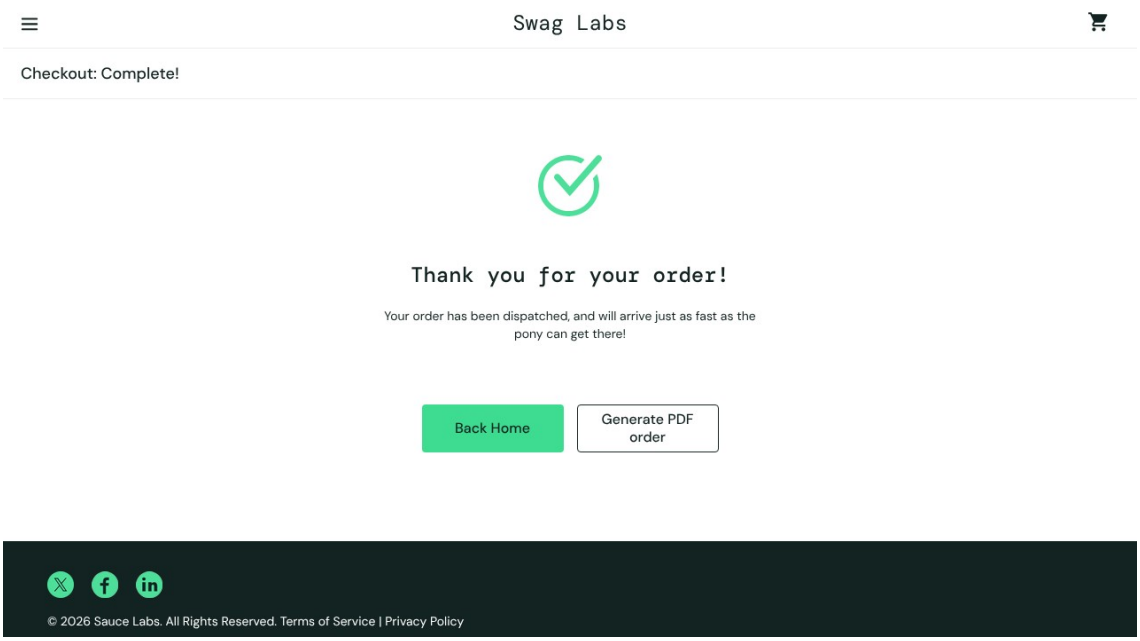
TC-005_01_campos_obligatorios.png



TC-005_02_apellido_en_nombre.png



TC-005_03_error_apellido.png



CTRL_checkout_standard_user_ok.png

5. Si el resultado es FAIL

DESVIACIÓN ENCONTRADA	El texto ingresado en Last Name se escribe en First Name, lo que impide continuar con la compra.

REFERENCIA DEL DEFECTO	BUG-001

6. Observaciones de ejecución

OBSERVACIONES	Recomendación: es el defecto más crítico encontrado porque impide finalizar una compra. Una vez corregido, se sugiere ejecutar de nuevo el flujo completo hasta la

	pantalla Thank you for your order! y agregar validaciones de longitud y caracteres especiales en los tres campos.

Reglas de uso

- El TC se diseña primero; el resultado observado se registra durante la ejecución.
- El resultado esperado debe estar definido antes de ejecutar.
- PASS = coincide; FAIL = diferencia verificable; BLOCKED = no se pudo ejecutar; NOT RUN = aún no ejecutado.
- Las evidencias deben permitir entender el resultado.
- El TC debe ser claro, concreto y reproducible por otro tester.

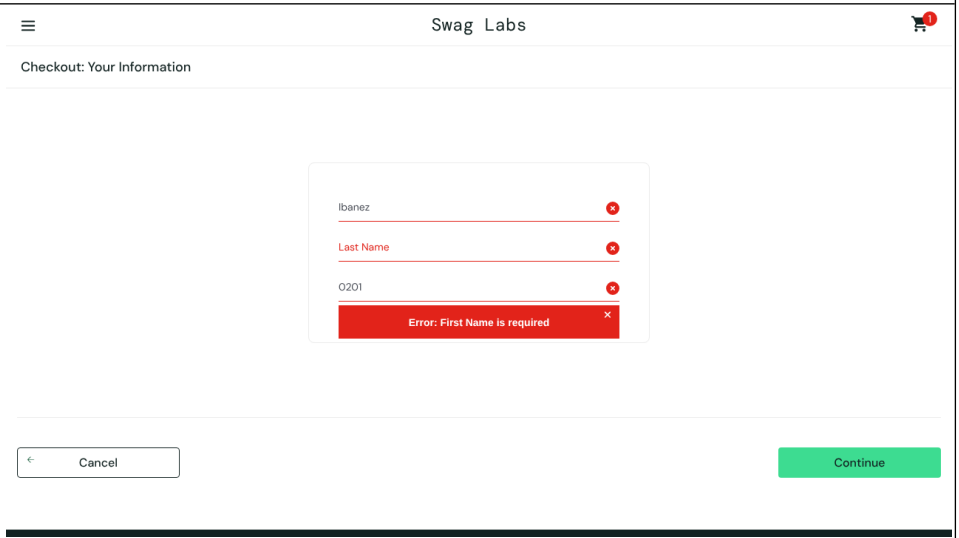
6. Bugs encontrados

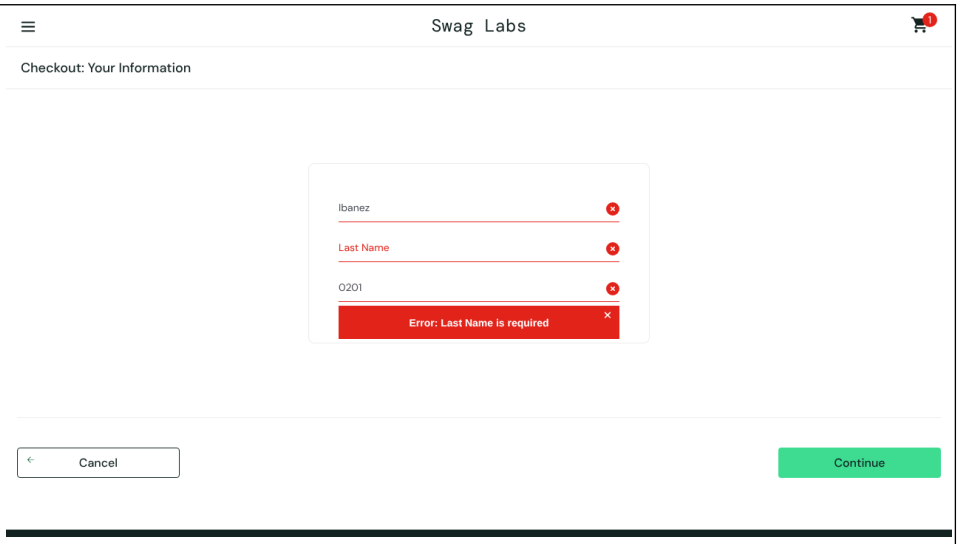
Durante la ejecución se encontraron dos defectos reproducibles. Ambos se registraron en la plantilla de bug report de la materia y como Issues con la etiqueta bug en el repositorio.

ID	Título	Caso	Severidad	Prioridad	Issue
BUG-001	El campo Last Name escribe el texto en First Name durante el checkout	TC-005	Crítica	Alta	#11
BUG-002	El ordenamiento por precio de menor a mayor no se aplica en el catálogo	TC-004	Mayor	Media	#12

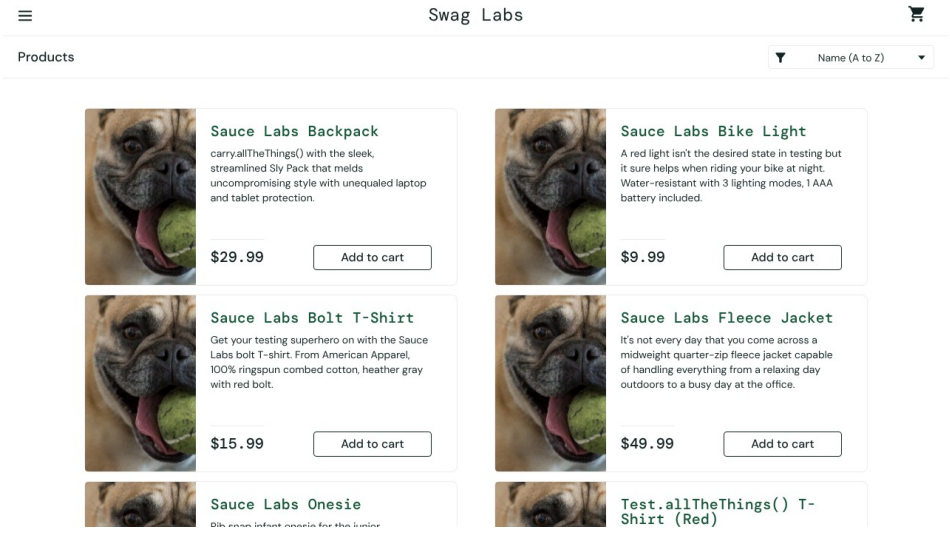
BUG-001 es el más grave porque impide completar una compra; BUG-002 afecta la búsqueda de productos, pero no bloquea el flujo principal. Los reportes completos, con sus evidencias, se presentan a continuación.

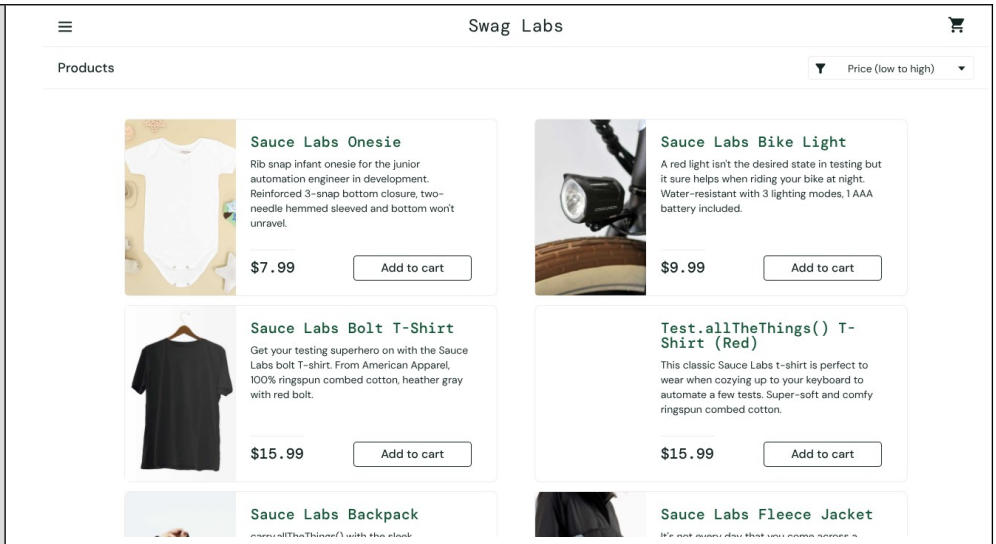
BUG REPORT TEMPLATE

#BUG-001	BUG REPORT
Title	El campo Last Name escribe el texto en First Name durante el checkout
Descripción	En el primer paso del checkout de https://www.saucedemo.com/ (Checkout: Your Information), al escribir en el campo Last Name con el usuario problem_user, el texto se registra en el campo First Name y reemplaza el nombre ya ingresado. Last Name queda vacío y el sistema impide continuar con la compra. Precondiciones: sesión iniciada con problem_user / secret_sauce y un producto en el carrito. Datos: First Name = Juan, Last Name = Ibanez, Zip/Postal Code = 0201.
Pasos	1.- Ingresar a https://www.saucedemo.com/ con problem_user / secret_sauce 2.- Presionar Add to cart en Sauce Labs Backpack 3.- Abrir el carrito y presionar Checkout 4.- Escribir Juan en First Name 5.- Escribir Ibanez en Last Name 6.- Escribir 0201 en Zip/Postal Code y presionar Continue Referencia: TC-005
RESULTADO ACTUAL	First Name muestra Ibanez, Last Name queda vacío y al presionar Continue aparece el mensaje "Error: Last Name is required". No es posible avanzar al resumen de la compra.
RESULTADO ESPERADO	Cada campo debe conservar su propio valor (First Name = Juan, Last Name = Ibanez) y, al presionar Continue, el sistema debe avanzar a Checkout: Overview.
PRIORIDAD	Alta 1-5 ➔ 1
SEVERIDAD	Crítica 1-5 ➔ 1
AMBIENTE	Dispositivo = PC/LAPTOP Navegador = Chromium 141 · resolución 1280x720 Usuario = problem_user Versión del software = No aplica (sitio de demostración)
ADJUNTOS	 *TC-005_02_apellido_en_nombre.png*

	 *TC-005_03_error_apellido.png*
COMENTARIOS	El defecto bloquea por completo el flujo de compra, por eso se clasifica como crítico. Con standard_user el mismo flujo termina correctamente, lo que ayuda a ubicar el problema en el manejo de los campos del formulario. Se recomienda revisar el enlace entre cada campo y su variable, y volver a ejecutar TC-005 después de la corrección.

BUG REPORT TEMPLATE

#BUG-002	BUG REPORT
Title	El ordenamiento por precio de menor a mayor no se aplica en el catálogo
Descripción	En el catálogo de productos de https://www.saucedemo.com/, al seleccionar la opción Price (low to high) con el usuario problem_user, los productos no cambian de orden y el filtro vuelve a mostrar Name (A to Z). Precondiciones: sesión iniciada con problem_user / secret_sauce en la página de productos. Datos: opción de ordenamiento Price (low to high).
Pasos	1.- Ingresar a https://www.saucedemo.com/ con problem_user / secret_sauce 2.- En la página Products, abrir el filtro de ordenamiento 3.- Seleccionar Price (low to high) 4.- Observar el orden de los precios Referencia: TC-004
RESULTADO ACTUAL	Los precios permanecen en el orden $29.99, $9.99, $15.99, $49.99, $7.99, $15.99 y el filtro muestra nuevamente Name (A to Z).
RESULTADO ESPERADO	Los productos deben mostrarse de menor a mayor precio ($7.99, $9.99, $15.99, $15.99, $29.99, $49.99) y el filtro debe conservar la opción seleccionada.
PRIORIDAD	Media 1-5 ➔ 3
SEVERIDAD	Mayor 1-5 ➔ 3
AMBIENTE	Dispositivo = PC/LAPTOP Navegador = Chromium 141 · resolución 1280x720 Usuario = problem_user Versión del software = No aplica (sitio de demostración)
ADJUNTOS	 *TC-004_01_orden_no_aplicado.png*

	 *CTRL_orden_standard_user_ok.png*
COMENTARIOS	El defecto no impide comprar, pero dificulta encontrar productos por precio y genera desconfianza en el usuario. Se recomienda probar también las demás opciones del filtro, ya que podrían estar afectadas por la misma causa.

7. Análisis de caja blanca

Se analizó la función validar_compra, que recibe la edad del cliente, el estado de su cuenta, el saldo disponible y el monto de la compra, y devuelve uno de cuatro mensajes posibles.

```
def validar_compra(edad, cuenta_activa, saldo, monto):
    if edad < 18:                          # D1
        return "Compra no permitida: edad"
    if not cuenta_activa:                  # D2
        return "Compra no permitida: cuenta"
    if saldo >= monto and monto > 0:       # D3
        return "Compra aprobada"
    return "Compra rechazada"
```

7.1 Decisiones y condiciones simples

Decisión	Expresión	Condiciones simples	Verdadera	Falsa
D1	edad < 18	C1: edad < 18	No permitida: edad	Pasa a D2
D2	not cuenta_activa	C2: cuenta inactiva	No permitida: cuenta	Pasa a D3
D3	saldo >= monto and monto > 0	C3: saldo >= monto C4: monto > 0	Compra aprobada	Compra rechazada

El código tiene 3 decisiones, 4 condiciones simples y 4 sentencias return, que son sus cuatro salidas posibles.

7.2 Criterio y mínimo de casos

Criterio: Se utilizó el criterio de cobertura de decisiones, que exige que cada decisión tome al menos una vez el valor verdadero y una vez el falso.

Cálculo: Las tres decisiones están en secuencia y las dos primeras terminan la función cuando son verdaderas. Por eso D1 verdadera, D2 verdadera, D3 verdadera y D3 falsa necesitan cada una su propio caso, mientras que los resultados falsos de D1 y D2 quedan cubiertos por los casos que avanzan. El mínimo es 4 casos, lo que coincide con la complejidad ciclomática V(G) = 3 decisiones + 1 = 4.

Comparación: La cobertura de sentencias también requiere 4 casos (uno por cada return). La cobertura de condiciones requiere 5, porque Python evalúa and en cortocircuito: para que C4 sea falsa, C3 debe ser verdadera, así que D3 falsa necesita dos casos distintos.

7.3 Casos diseñados

ID	edad	cuenta	saldo	monto	Resultado esperado	Cubre
CB-01	16	True	500	200	Compra no permitida: edad	D1 = V
CB-02	25	False	500	200	Compra no permitida: cuenta	D1 = F, D2 = V
CB-03	30	True	500	200	Compra aprobada	D1 = F, D2 = F, D3 = V

Con los tres casos se cubren 5 de los 6 resultados de decisión (83 %): D1 y D2 toman ambos valores, pero D3 nunca es falsa y la sentencia return "Compra rechazada" no se ejecuta.

7.4 Ajuste necesario

Los tres casos no alcanzan el criterio. Para completarlo se agrega un cuarto caso que haga falsa a D3:

ID	edad	cuenta	saldo	monto	Resultado esperado	Cubre
CB-04	30	True	100	300	Compra rechazada	D3 = F (C3 = F)
CB-05*	30	True	500	0	Compra rechazada	C3 = V, C4 = F

Con CB-01 a CB-04 se logra el 100 % de cobertura de decisiones y de sentencias. *CB-05 solo es necesario si se busca además la cobertura de condiciones.

7.5 Comprobación con coverage.py

Los casos se automatizaron con pytest y se midió la cobertura de ramas con coverage run --branch. La medición confirma el análisis manual:

Conjunto de casos	Sentencias sin ejecutar	Ramas parciales	Cobertura
CB-01 a CB-03	1 (línea 11)	1	86 %
CB-01 a CB-04	0	0	100 %
CB-01 a CB-05	0	0	100 %

```
Medición de cobertura con coverage.py (--branch) · 20/10/2026

=== 3 casos diseñados (CB-01 a CB-03)
$ python -m coverage run --branch -m pytest test_compra_3_casos.py
3 passed in 0.02s
Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
compra.py       8      1      6      1    86%   11
-----------------------------------------------------------
TOTAL           8      1      6      1    86%
```

Observación: Con monto = 0 la función devuelve el mismo mensaje que con saldo insuficiente. Se recomienda un mensaje distinto para montos inválidos, de modo que el usuario entienda por qué se rechazó la compra.

8. Escenarios Gherkin

Como actividad adicional se describieron en Gherkin los flujos de inicio de sesión y de checkout. Se incluyen escenarios exitosos, uno de error y un Scenario Outline que valida los tres campos obligatorios del checkout. El archivo docs/saucedemo.feature del repositorio contiene la versión lista para usar.

```
Feature: Inicio de sesión en SauceDemo
  Como cliente de Swag Labs
  quiero iniciar sesión con mi usuario
  para acceder al catálogo de productos

  Background:
    Given que estoy en la página de login de "https://www.saucedemo.com/"

  Scenario: Acceso exitoso con un usuario habilitado
    When ingreso el usuario "standard_user" y la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería ver la página "Products"
    And el catálogo debería mostrar 6 productos

  Scenario: Acceso denegado a un usuario bloqueado
    When ingreso el usuario "locked_out_user" y la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería permanecer en la página de login
    And debería ver el mensaje "Epic sadface: Sorry, this user has been locked out."


Feature: Datos del comprador en el checkout
  Como cliente con productos en el carrito
  quiero ingresar mis datos de envío
  para continuar con la compra

  Background:
    Given que inicié sesión como "standard_user"
    And tengo el producto "Sauce Labs Backpack" en el carrito
    And estoy en la página "Checkout: Your Information"

  Scenario: Continuar con todos los datos completos
    When ingreso "Juan" en "First Name"
    And ingreso "Ibanez" en "Last Name"
    And ingreso "0201" en "Zip/Postal Code"
    And presiono el botón "Continue"
    Then debería ver la página "Checkout: Overview"

  Scenario Outline: No continuar si falta un dato obligatorio
    When ingreso "<nombre>" en "First Name"
    And ingreso "<apellido>" en "Last Name"
    And ingreso "<postal>" en "Zip/Postal Code"
    And presiono el botón "Continue"
    Then debería ver el mensaje "<mensaje>"
    But no debería avanzar a "Checkout: Overview"

    Examples:
      | nombre | apellido | postal | mensaje                            |
      |        | Ibanez   | 0201   | Error: First Name is required      |
      | Juan   |          | 0201   | Error: Last Name is required       |
      | Juan   | Ibanez   |        | Error: Postal Code is required     |
```

Cada escenario parte de un estado conocido (Given), ejecuta una acción del usuario (When) y verifica un resultado observable en pantalla (Then). Los mensajes de error del Scenario Outline se comprobaron en la aplicación.

9. Organización del trabajo en Git y GitHub

Repositorio: https://github.com/aJuanga/Examen-Hito-3-TQA

Project Board: https://github.com/users/aJuanga/projects/3

El repositorio se creó en GitHub y se gestionó desde la terminal con Git. Los archivos se subieron en commits separados por actividad, y cada mensaje de commit cierra el Issue correspondiente:

```
git init -b main
git remote add origin https://github.com/aJuanga/Examen-Hito-3-TQA.git
git add README.md .gitignore
git commit -m "Estructura inicial del repositorio y README" -m "Closes #8"
git add docs/casos_caja_negra.md docs/plantillas
git commit -m "Diseño y ejecución de los 5 casos de caja negra (TC-001 a TC-005)"
...
git push -u origin main
```

Commit	Mensaje	Issues
9f1e73a	Estructura inicial del repositorio y README	#8
86cda92	Diseño y ejecución de los 5 casos de caja negra (TC-001 a TC-005)	#1, #3
ad1adc9	Evidencias de ejecución en SauceDemo	#2, #5
255215e	Registro de BUG-001 y BUG-002 en la plantilla de bugs	#4 (ref. #11, #12)
799021e	Análisis de caja blanca de validar_compra y medición de cobertura	#6
e3241ca	Escenarios Gherkin de inicio de sesión y checkout	#7
—	Informe final en PDF	#9

Se registraron 12 Issues: 10 tareas (etiquetas tarea y documentacion) y 2 bugs. El Board tiene las columnas To Do, In Progress, In Review y Done. Las tareas terminadas pasaron a Done al cerrarse con los commits; los bugs permanecen en To Do hasta que el equipo de desarrollo los corrija.

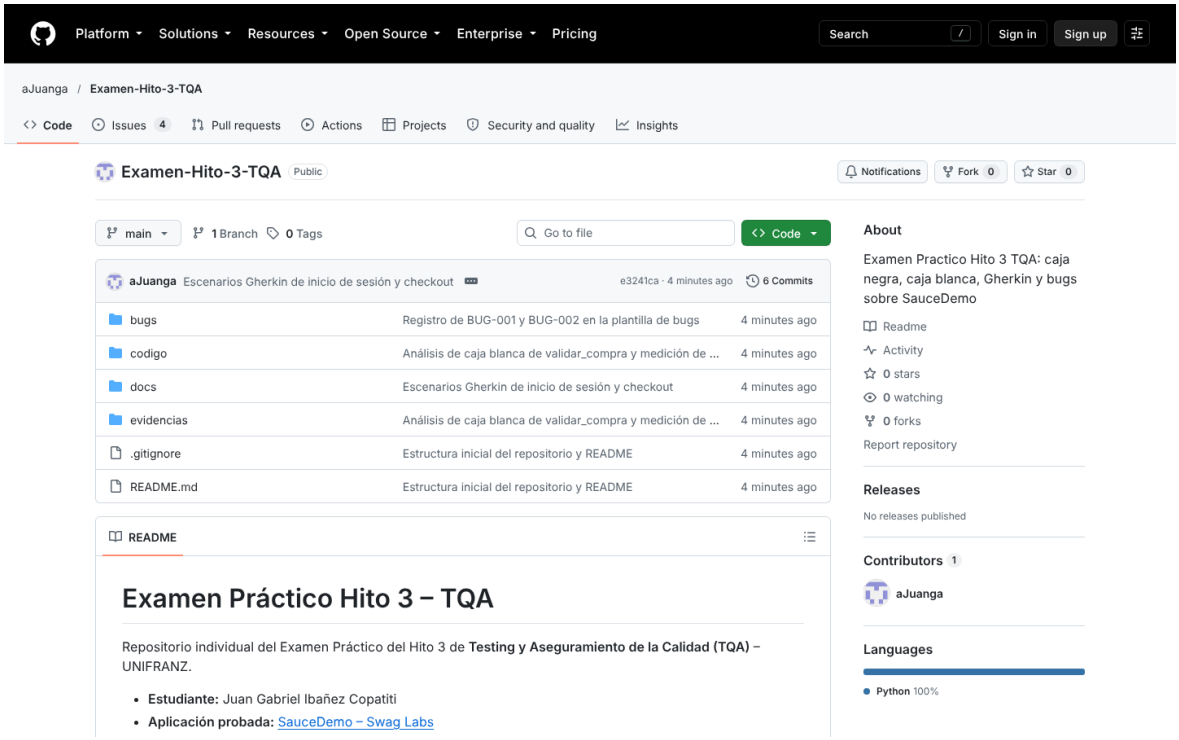


Figura 1. Repositorio Examen-Hito-3-TQA con su estructura de carpetas.

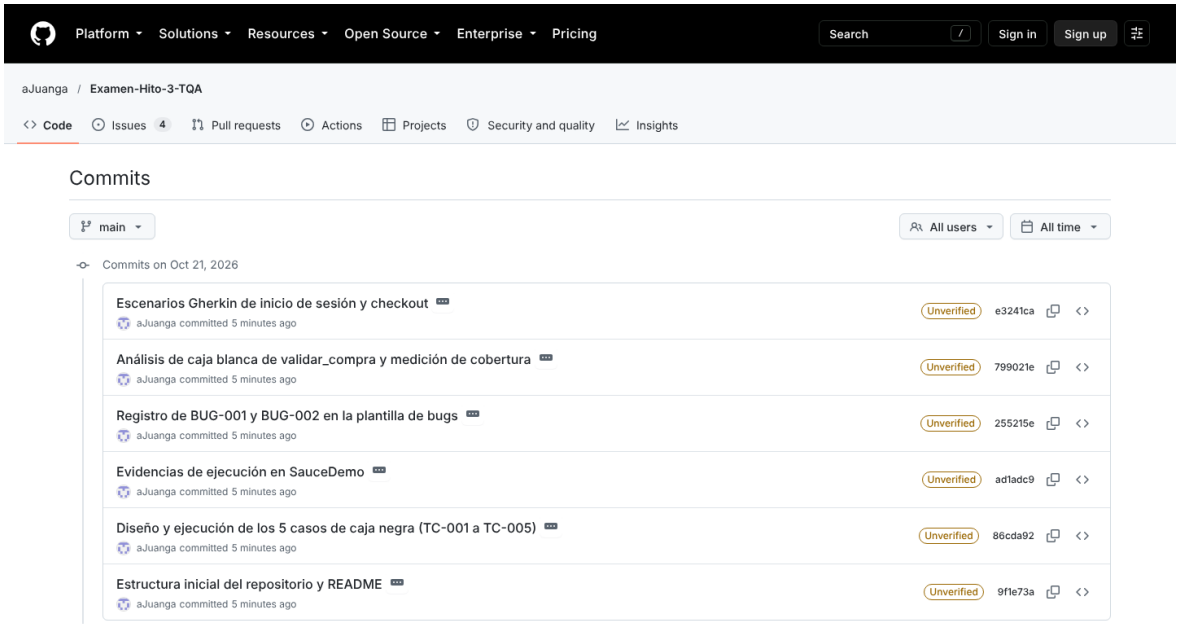


Figura 2. Historial de commits realizados desde la terminal.

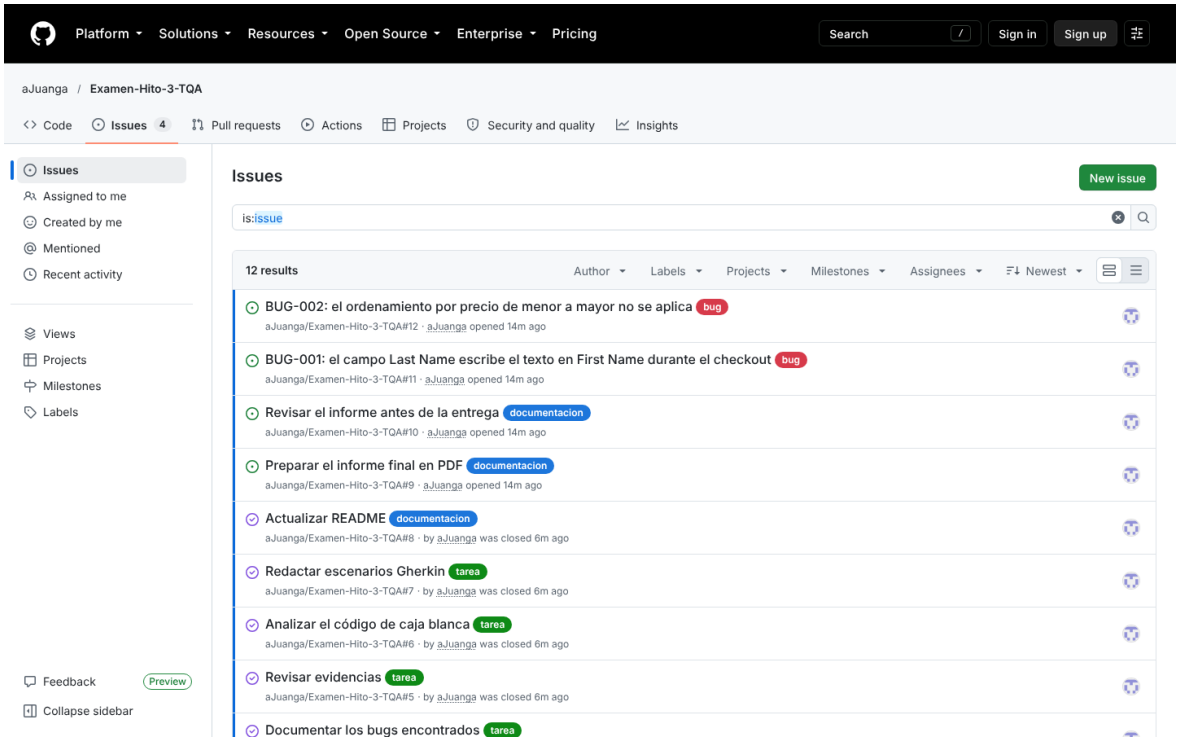


Figura 3. Issues de tareas y bugs con sus etiquetas.

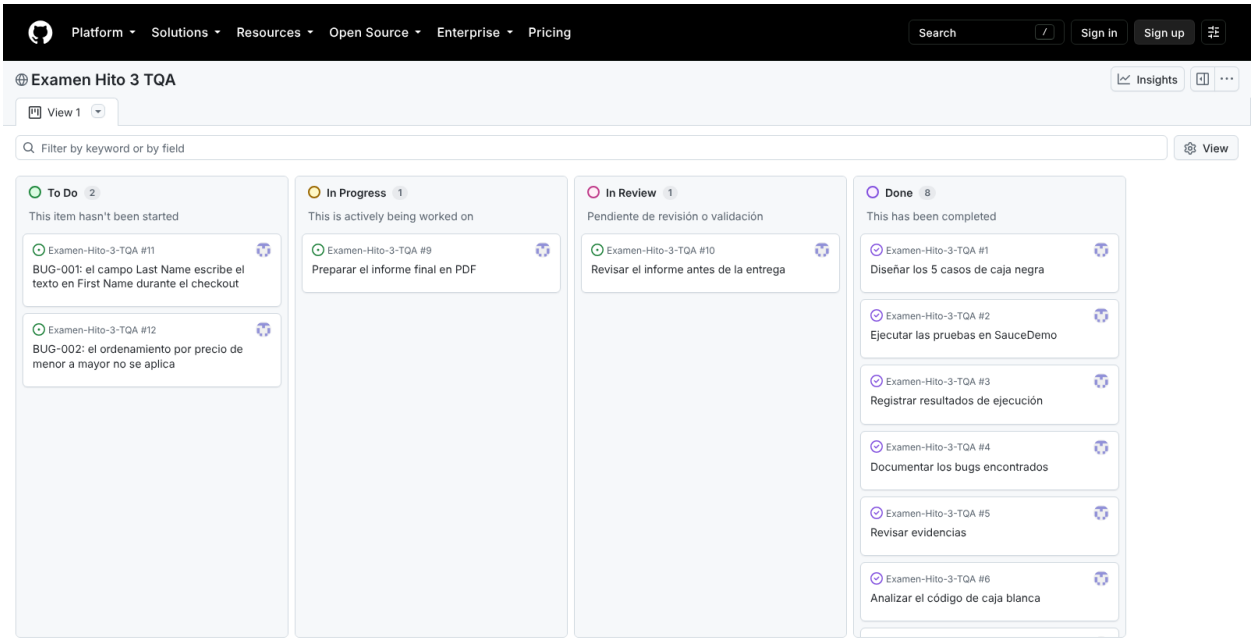


Figura 4. GitHub Project Board con las columnas To Do, In Progress, In Review y Done.

10. Conclusiones y recomendaciones

- **Caja negra.** Los cinco casos permitieron recorrer los flujos principales de SauceDemo. Tres pasaron y dos fallaron; los fallos solo aparecen con problem_user, lo que demuestra que el mismo flujo puede comportarse distinto según el usuario y que conviene probar con varios perfiles.

- **Bugs.** BUG-001 bloquea la compra y debe corregirse primero. Repetir el flujo con standard_user como control ayudó a describir el defecto con precisión y a descartar errores de ejecución.
- **Caja blanca.** El mínimo de casos depende del criterio: 4 para decisiones y sentencias, y 5 para condiciones por el cortocircuito de and. Los 3 casos pedidos cubren el 83 % de las decisiones, y medir con coverage.py confirmó qué rama faltaba.
- **Gherkin.** Escribir los escenarios obligó a definir resultados verificables. El Scenario Outline resume en una sola estructura tres validaciones del formulario.
- **GitHub.** Relacionar commits, Issues y Board dejó trazabilidad entre cada tarea, su archivo y su evidencia.
- **Recomendación.** Volver a ejecutar TC-004 y TC-005 como pruebas de regresión cuando se corrijan los defectos, y automatizar los escenarios Gherkin con una herramienta como Behave o Cucumber para repetirlos en cada versión.

11. Evidencias

Todas las evidencias están en la carpeta evidencias/ del repositorio:

Archivo	Relacionado con
TC-001_01_credenciales.png	TC-001
TC-001_02_catalogo.png	TC-001
TC-002_01_usuario_bloqueado.png	TC-002
TC-003_01_dos_productos.png	TC-003
TC-003_02_carrito.png	TC-003
TC-003_03_producto_eliminado.png	TC-003
TC-004_01_orden_no_aplicado.png	TC-004
CTRL_orden_standard_user_ok.png	TC-004
TC-005_01_campos_obligatorios.png	TC-005
TC-005_02_apellido_en_nombre.png	TC-005
TC-005_03_error_apellido.png	TC-005
CTRL_checkout_standard_user_ok.png	TC-005
cobertura_caja_blanca.txt	Caja blanca
GH_01 a GH_04 (evidencias/github/)	Git y GitHub